DUNKIN KSA
AI CHATBOT

ZERO TO HERO — explained like you're brand new to coding

In simple words: We built a website where a customer can ask Dunkin questions. A Python backend handles the question, menu data provides product information, and Gemini can generate an answer using information retrieved from ChromaDB. Then we saved the project in a private GitHub repository.

This guide explains what we did, why each part exists, how the parts connect, what is complete, and what is still only a demo.

Project checkpoint

• Frontend: HTML, CSS and JavaScript

• Backend: Python, FastAPI, Uvicorn and Pydantic

• AI: Google Gemini generation and embeddings

• Retrieval: ChromaDB and a RAG pipeline

• Data: Dunkin KSA menu stored as JSON

• Features: menu search, category/price search, outlet information and complaint conversation flow

• Packaging: Dockerfiles and Docker Compose configuration

• Version control: Git, private GitHub repository and README documentation

Important: this is a prototype. Complaint/order verification is not connected to a real billing system, conversation state is in memory, and Docker deployment should be tested end-to-end before production use.

1. The whole project in one picture

Imagine a Dunkin shop. The customer is at the counter, the website is the counter, FastAPI is the waiter, the menu JSON is the menu book, ChromaDB is a searchable filing cabinet, and Gemini turns relevant information into a natural-sounding answer.

```
CUSTOMER
   |
   v
FRONTEND: HTML + CSS + JavaScript
   |  sends a message as an HTTP request / JSON
   v
BACKEND: FastAPI in Python (main.py)
   |
   +--> Intent detection
   +--> Direct menu/category/price search --> dunkin_menu.json
   +--> Outlet information --> official store locator link
   +--> Complaint workflow --> conversation state
   |
   +--> RAG path (for questions needing semantic retrieval)
          |
          v
      Gemini embedding model
          |
          v
      ChromaDB vector search
          |
          v
      Relevant menu context
          |
          v
      Gemini generation model
   |
   v
JSON response --> Frontend displays answer
```

The backend is a traffic controller. It decides whether Python can answer with a direct lookup or whether the question should go through the RAG/LLM path.

2. What we built, step by step

Step 1 — Chose a real use case

We started with an AI chatbot idea and focused it on Dunkin KSA customer service. Examples include menu questions, prices, categories, outlet information and a guided complaint conversation. A specific use case tells us what data and features the bot needs.

Step 2 — Set up the development environment

Python runs backend code. The virtual environment (.venv) keeps this project's Python packages separate. pip installs packages. Git tracks code history. We used Windows PowerShell to run commands.

Step 3 — Built the frontend

• **HTML** is the structure: chat window, text box, buttons and headings.

• **CSS** is the appearance: colors, spacing, layout, responsive styles and dark mode.

• **JavaScript** makes the page interactive: send messages, call the backend, and display answers and clickable options.

Shop analogy: HTML builds the counter, CSS decorates it, and JavaScript lets the counter do things.

Step 4 — Built the backend API

FastAPI creates HTTP endpoints. The frontend sends a request to /chat. Pydantic validates request fields, including session_id and message. Uvicorn runs the API server. The /docs page let us test the endpoint in a browser.

```
Example request (simplified):
```

```
{
  "session_id": "a-conversation-id",
  "message": "How much is Mango Juice Sante?"
}
```

Shop analogy: FastAPI is the waiter carrying the customer's question to the right place and returning the answer.

Step 5 — Collected and structured menu data

We used scripts to inspect and process Dunkin KSA website information. The processed data lives in backend/dunkin_menu.json. At the time we built the vector database, it contained 61 menu items. Each item can have fields such as name, price, currency, category and source.

```
Example:
{
  "name": "CHOCOLATE MUFFIN",
  "price": 7.0,
  "currency": "SAR",
  "category": "Bakery"
}
```

JSON is a structured text format: think of it as a machine-readable menu spreadsheet. Product data should be checked because missing values or placeholder fields are not reliable facts.

Step 6 — Added normal menu functions

We added Python functions for category browsing, text search, donut listing and filtering by maximum price. Exact product lookups often do not need an LLM. A confirmed product and price can be retrieved directly from JSON.

Step 7 — Added embeddings and ChromaDB

An embedding model turns text into a list of numbers representing aspects of its meaning. ChromaDB stores those vectors with documents and searches for vectors close to a customer's query. This helps match related wording even when exact words differ.

We used gemini-embedding-001 and a ChromaDB collection named dunkin_menu_ksa. The build completed for 61 menu items. The generated vector database is local and excluded from Git; it can be recreated by running the vector database script with valid configuration.

Step 8 — Added RAG

RAG means Retrieval-Augmented Generation. Retrieve relevant data, augment the prompt with that data, and generate a response. Our pipeline embeds a question, retrieves relevant menu documents from ChromaDB, adds those documents to the prompt, and asks Gemini to write the answer.

```
Customer question
      |
      v
Create query embedding
      |
      v
Search ChromaDB
      |
      v
Retrieve relevant menu documents
      |
      v
Put retrieved data + question into prompt
      |
      v
Gemini generates the response
```

We used gemini-3.5-flash-lite for text generation. RAG can reduce unsupported guesses, but it does not guarantee correctness. Exact prices should use verified structured records when possible, and the bot should say when information is missing.

Step 9 — Added outlet and complaint flows

The outlet service provides the official store locator and avoids inventing unverified opening hours. The complaint flow asks the customer to choose an issue type; order complaints can proceed to a billing-number prompt. The billing step is a demonstration only; it does not query a real order database or CCTV system.

Step 10 — Added session state

The frontend creates a session ID and sends it with each message. The backend uses it to associate a conversation with its current complaint step. State is held in a Python dictionary, so it disappears when the backend restarts and is not suitable for a multi-worker production setup. Persistent storage such as SQLite or PostgreSQL would be a later improvement.

Step 11 — Prepared Docker

We created a backend Dockerfile, a frontend Dockerfile and docker-compose.yml. The intended setup runs FastAPI in one container and serves the frontend with Nginx in another. Docker configuration exists, but test a fresh docker compose up --build end-to-end before treating container deployment as verified.

Step 12 — Uploaded to private GitHub

We created a private repository, added .gitignore, initialized Git, staged and committed the project, connected the remote, authenticated with GitHub CLI, and pushed the code. Then we wrote README.md and pushed a second commit.

```
First commit:  e5b7221  Initial Dunkin KSA AI chatbot
Second commit: 18f28ba  Add project documentation
```

Git is the local version-history tool. GitHub hosts the remote repository. A commit is a saved snapshot; push uploads commits. The README is the project's instruction page.

3. The project files — what they mean

Path	Simple job
frontend/index.html	The page structure
frontend/style.css	Colors, layout and styling
frontend/app.js	Sending messages and showing responses
backend/main.py	Main API and chatbot routing
backend/chatbot/intent.py	Detects common intent types
backend/menu/service.py	Searches categories, names and prices
backend/outlet/service.py	Outlet-related responses and locator
backend/dunkin_menu.json	Structured KSA menu records
backend/create_vector_db.py	Creates embeddings and populates ChromaDB
backend/search_menu.py	Tests semantic retrieval
backend/rag_chat.py	Standalone RAG demonstration
backend/requirements.txt	Python package dependencies
backend/Dockerfile	Build instructions for backend container
frontend/Dockerfile	Build instructions for frontend container
docker-compose.yml	Runs containers together
.env	Local secrets/configuration; do not commit
.gitignore	Files Git should ignore

Path	Simple job
README.md	Project documentation

4. Technologies in donkey language

Technology	Think of it as…
HTML	The skeleton of the shop counter
CSS	The paint, decoration and layout
JavaScript	The hands that make the page interactive
Python	The language for backend instructions
FastAPI	The waiter/API that accepts requests
Uvicorn	The engine that runs the API server
Pydantic	The form checker for required fields
JSON	An organized menu notebook
Embedding model	Turns text into numerical representations
ChromaDB	A searchable filing cabinet for vectors/documents
Gemini LLM	Writes a natural-language response
RAG	Look up information first, then ask AI to answer
Git	A local history book of code changes
GitHub	The online home for the Git repository
Docker	A package/container for running an app consistently
Docker Compose	The manager that starts multiple containers
.gitignore	A list of things Git should leave out
README.md	The instruction manual for the project

5. What is done vs. what is not done

Built / configured

• Chat UI and message exchange with FastAPI.

• KSA menu JSON and Python menu functions.

• Gemini embeddings and ChromaDB collection built for 61 menu items.

• Standalone semantic search and RAG test scripts.

• Main chatbot routes for menu, outlet and complaint flows.

• Session ID sent by the frontend and demo in-memory conversation state.

• Dockerfiles and Docker Compose configuration.

• Private GitHub repository with README and two commits.

Still a prototype / needs verification

• Billing number is not checked against a real order system.

• Conversation state is not persisted in a database.

• Menu values and product attributes require validation; placeholder values must not be presented as facts.

• The frontend API URL and CORS settings should be adjusted for the target deployment.

• Docker needs a fresh end-to-end test after current code changes.

• Production needs stronger security, rate limits, logging, monitoring and error handling.

• Arabic support, FAQ coverage, analytics and feedback collection are possible future features.

6. Protect your API key

Your Gemini API key belongs in the root .env file, not in source code or GitHub. The current .gitignore excludes .env. Before every commit, inspect git status and be careful with files containing secrets. If a key was ever exposed in a screenshot or committed, revoke it and create a replacement. A private repository is not a safe place for secrets.

7. Everyday Git workflow

```
# From the project root
git status
git add .
git commit -m "Describe the change"
git push
```

Simple meaning: check what changed, choose what to save, create a named snapshot, then upload it to GitHub. Before git add ., check that no secrets or unwanted generated files are included.

8. Quick revision

• **What is the frontend?** The part the customer sees and clicks.

• **What is the backend?** Server-side code that receives requests and decides how to answer.

• **What is RAG?** Retrieve relevant information first, then provide it to the LLM to generate a response.

• **What is ChromaDB?** The vector database used to find semantically relevant documents.

• **Commit vs. push?** Commit saves a local snapshot; push uploads commits to the remote GitHub repository.

ONE SENTENCE TO REMEMBER: We built a web chat interface, connected it to a Python API, added menu data and a retrieval-plus-Gemini answer path, implemented basic service workflows, prepared Docker packaging, and saved the source code in a private GitHub repository.

End of guide · Use this as a learning reference, not as a claim that every production feature is complete.